

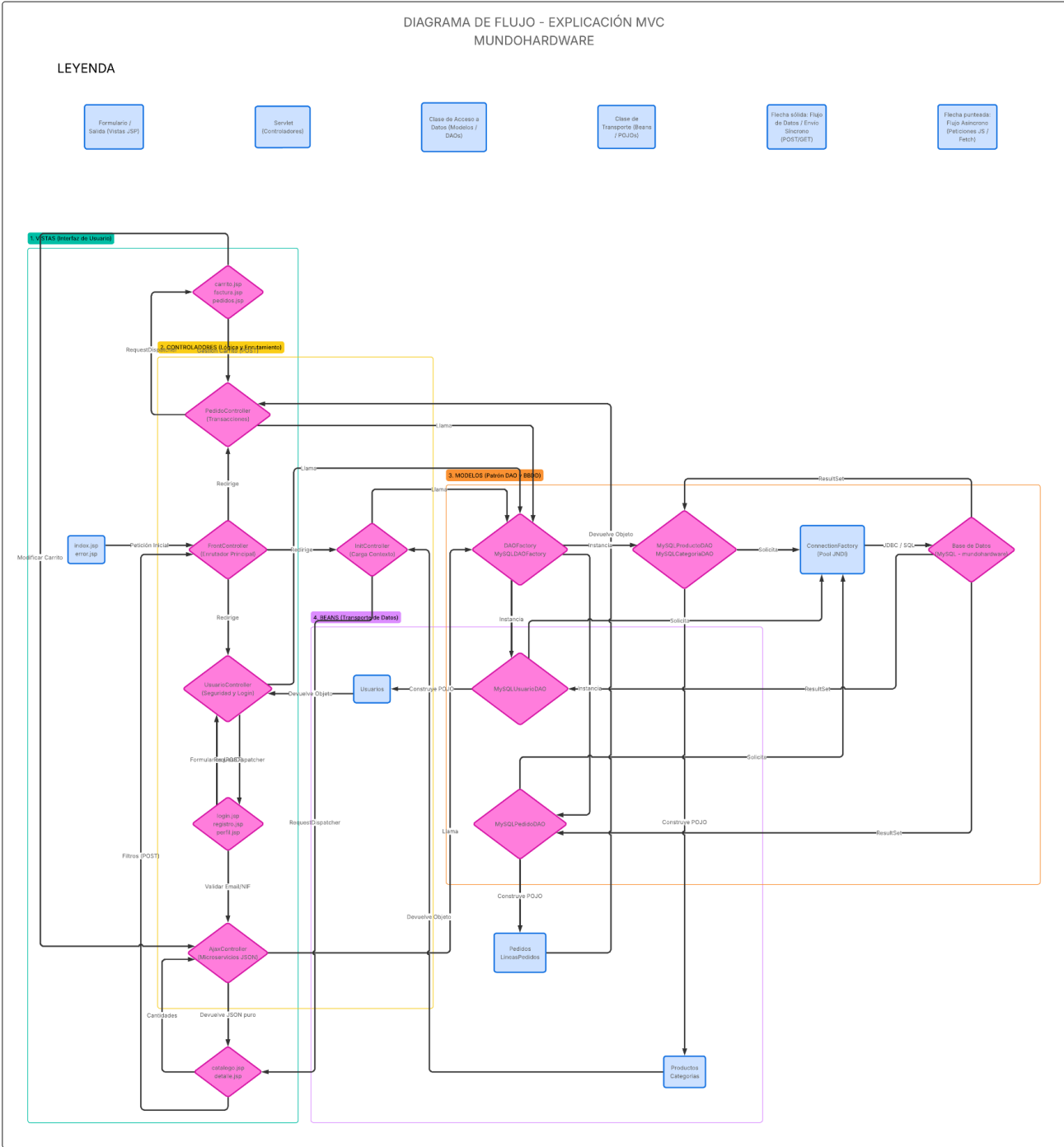
DIAGRAMA DE FLUJO - MUNDOHARDWARE



Alberto García Izquierdo

DAW2ºA 25/26

DWESV



DOCUMENTACIÓN TÉCNICA DE ARQUITECTURA MVC: MUNDOHARDWARE

LEYENDA

- Formulario (Vista): Interfaz gráfica orientada a la entrada y captura de datos del cliente.
- Servlet (Controlador): Módulo de enrutamiento y procesamiento lógico de las peticiones HTTP.
- Clase (Modelo): Capa de abstracción de datos (DAO) encargada de la lógica de negocio y persistencia.
- Clase (Beans): Objeto de transferencia de datos (POJO/DTO) para la encapsulación de estado.
- Salida (Vista): Interfaz final para la renderización y presentación de la información.
- Flujo: Dirección de la ejecución, delegación de tareas y transferencia de control.
- Carga: Proceso de inicialización de datos en la memoria de la aplicación.
- Utiliza: Relación de dependencia funcional entre los distintos componentes del sistema.

DOCUMENTOS

Modelos (Capa de Acceso a Datos)

- MySQLUsuarioDAO – Ejecuta las transacciones relativas a la autenticación, registro y validación criptográfica de credenciales de los usuarios.
- MySQLProductoDAO – Gestiona las consultas de selección, filtrado multidimensional y búsqueda dinámica sobre el catálogo de artículos.
- MySQLPedidoDAO – Administra la persistencia de los carritos de compra y consolida las transacciones financieras mediante el control estricto de bloqueos (Commit/Rollback).
- MySQLCategoriaDAO – Responsable de la extracción y mapeo de las familias taxonómicas de los productos.

- ConnectionFactory – Implementa la gestión optimizada de las conexiones al motor relacional mediante un recurso de tipo Pool (JNDI).
- DAOFactory / MySQLDAOFactory – Aplica el patrón de diseño abstracto Factory para la instanciación escalable y segura de las clases de acceso a datos.

Vistas

- index.jsp – Punto de entrada que enruta la aplicación hacia el catálogo; estructurado de forma modular mediante la inclusión de cabecera.jsp, pie.jsp y metas.inc.
- catalogo.jsp – Entorno de visualización del inventario, integrando filtros interactivos y renderizado dinámico.
- detalle.jsp – Interfaz extendida para la exposición pormenorizada de las especificaciones técnicas de un producto aislado.
- carrito.jsp – Módulo de visualización y manipulación asíncrona de las líneas de pedido en curso.
- login.jsp / registro.jsp – Formularios de seguridad para el control de acceso y alta, implementando validaciones de integridad en el lado del cliente.
- perfil.jsp / pedidos.jsp – Panel de administración privado para la edición de información personal y auditoría del histórico de transacciones.
- factura.jsp – Documento de confirmación que detalla el resumen de la operación comercial y su desglose impositivo.
- error.jsp / aviso.jsp – Vistas genéricas de intercepción para la notificación controlada de excepciones, errores de servidor o mensajes de estado.

Controladores

- FrontController – Servlet principal que actúa como punto único de entrada, interceptando las peticiones y delegando el flujo de ejecución a los sub-controladores pertinentes.
- InitController – Orquesta el ciclo de vida de arranque, inyectando el catálogo y los atributos estáticos en el contexto global de la aplicación.

- UsuarioController – Centraliza el control de sesiones, autorización de acceso, modificación de perfiles y la gestión multiparte de archivos binarios (avatares).
- PedidoController – Coordina la persistencia temporal del carrito (Sesión/Cookies) y su consolidación definitiva en la base de datos tras la confirmación de facturación.
- AjaxController – Controlador especializado en la resolución de microservicios asíncronos en segundo plano, emitiendo respuestas estructuradas nativamente en formato JSON.

Excepciones

- SQLException – Evento de error capturado y controlado ante contingencias en el motor relacional, bloqueos transaccionales o pérdidas de integridad en la conexión a la base de datos.

Beans

- Usuarios – Estructura de almacenamiento de las credenciales y atributos demográficos del cliente.
- Productos / Categorías – Entidades que encapsulan la definición comercial y catalogación del inventario.
- Pedidos / LíneasPedidos – Modelado de las transacciones, agregando los cálculos económicos y la cardinalidad de los productos adquiridos.

LLAMADAS

- De JSP a controlador – Método HTTP POST orientado a la transmisión segura del formulario (definiendo la ruta de ejecución mediante el parámetro oculto accion).
- De JSP a controlador asíncrono – Utilización de la API Fetch (JavaScript) para la invocación de procesos en segundo plano sin recarga de interfaz.
- De controlador a modelos – Invocación de la lógica de persistencia abstrayendo la conexión a través de la interfaz del DAOFactory.

- De controlador a JSP – Despacho del flujo y transferencia de peticiones empleando el método `getRequestDispatcher(url).forward(request, response)`.
- De JSP al controlador final – Petición estándar HTTP GET generada a través de hipervínculos.

FORMAS DE DECLARAR UN CONTROLADOR (SERVLET)

- Definición estática centralizada en el descriptor de despliegue (`web.xml`).
- Configuración dinámica en tiempo de compilación mediante anotaciones de la API Servlet (`@WebServlet`, `@MultipartConfig`).

PASO DE DATOS ENTRE LOS DIFERENTES COMPONENTES

- De vistas a controlador – Transmisión codificada vía POST, GET o cuerpo AJAX.
- De controlador a modelos – Inyección por parámetros de los métodos expuestos; el resultado de la consulta poblacional retorna instanciando un Bean o una colección iterativa de estos.
- Modelos a controlador – Retorno tipado que refleja el estado de la transacción (instancias de Objetos, estructuras List, o primitivos lógicos y numéricos).
- De controlador a JSP – Delegación de la información de negocio mediante fijación de atributos en los objetos de alcance:
 - `request.setAttribute(String, Object)`
 - `session.setAttribute(String, Object)`
- En los JSP se leen estos atributos de petición con – Resolución embebida mediante el Lenguaje de Expresiones de Java (`${atributo}`).

BEANS

- Arquitectura estandarizada POJO (Plain Old Java Object).
- Implementación estricta de la interfaz `Serializable` para asegurar su viabilidad en la persistencia de sesiones.
- Encapsulación de datos mediante la declaración de atributos de acceso `private`.
- Definición de constructores predeterminados (implícitos) y sobrecargados (explícitos).

- Acceso indirecto al estado del objeto a través de métodos accesorios y mutadores (getters y setters).

MÉTODOS PARA INTERPRETAR UN FORMULARIO EN UN SERVLET

- `String getParameter(String name)`
- `String[] getParameterValues(String name)`
- `Map<String, String[]> getParameterMap()`
- `Enumeration<String> getParameterNames()`
- Integración de API de Terceros: Automatización de la carga de objetos empleando `BeanUtils.populate(Object, Map)` de Apache Commons.

INCLUDES

- Inyección dinámica de componentes estructurales y cabeceras de metadatos para la renderización modular de la vista: `cabecera.jsp`, `metas.inc` y `pie.jsp`.